12 — Advanced Classical ML: Production-Ready Techniques

Time: ~4-5 hours | **Level**: Intermediate → Advanced

What you'll learn:

- Bias-variance tradeoff: diagnosing with learning curves
- Gradient boosting: XGBoost and LightGBM comparison
- Feature engineering: strategies that win competitions
- Handling imbalanced data: SMOTE, class weights, threshold tuning
- Hyperparameter optimization with Optuna
- Model interpretability with SHAP
- Production pipelines with sklearn

Prerequisites: Notebook 02 (Classical ML), Notebook 11 (Math foundations)

Beyond `model.fit()`: Real-World ML

Getting 95% accuracy on a clean dataset is the tutorial. Getting 87% accuracy on messy, imbalanced, real-world data with explainable predictions is the job.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_classification, fetch_california_housing
from sklearn.model_selection import train_test_split, learning_curve, cross_
from sklearn.metrics import accuracy_score, f1_score, classification_report,
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassif
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
```

1. Bias-Variance Tradeoff: Diagnosing with Learning Curves

Symptom	Diagnosis	Fix
Train ↑, Val ↑, big gap	**High variance** (overfitting)	More data, regularization, simpler model

Symptom	Diagnosis	Fix
Train ↓, Val ↓, small gap	**High bias** (underfitting)	More features, complex model, less regularization
Train ↑, Val ↑, small gap	**Good fit**	You're done (or need more data for both)

In [ ]:

```
# —— Learning curves for bias–variance diagnosis ————————————————

X, y = make_classification(n_samples=2000, n_features=20, n_informative=10,
                           n_redundant=5, random_state=42)

models = {
    'High Bias (Logistic Regression)': LogisticRegression(max_iter=1000),
    'Good Balance (Random Forest)': RandomForestClassifier(n_estimators=100,
    'High Variance (Deep RF)': RandomForestClassifier(n_estimators=100, max_
}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, model) in zip(axes, models.items()):
    train_sizes, train_scores, val_scores = learning_curve(
        model, X, y, train_sizes=np.linspace(0.1, 1.0, 10),
        cv=5, scoring='accuracy', n_jobs=-1
    )
    ax.plot(train_sizes, train_scores.mean(axis=1), 'o-', label='Train')
    ax.fill_between(train_sizes, train_scores.mean(axis=1) - train_scores.st
                    train_scores.mean(axis=1) + train_scores.std(axis=1), al
    ax.plot(train_sizes, val_scores.mean(axis=1), 'o-', label='Validation')
    ax.fill_between(train_sizes, val_scores.mean(axis=1) - val_scores.std(ax
                    val_scores.mean(axis=1) + val_scores.std(axis=1), alpha=
    ax.set_xlabel('Training Size')
    ax.set_ylabel('Accuracy')
    ax.set_title(name)
    ax.legend()
    ax.set_ylim(0.6, 1.05)

plt.suptitle('Learning Curves: Diagnosing Bias vs Variance', fontsize=14)
plt.tight_layout()
plt.show()
```

2. Gradient Boosting: XGBoost & LightGBM

Feature	XGBoost	LightGBM
Tree growth	Level-wise	**Leaf-wise** (faster)
Speed	Fast	**Faster**
Memory	More	**Less**
Categorical	Needs encoding	**Native support**
Best for	Structured data (Kaggle winner)	Large datasets

In [ ]:

```
# ── XGBoost vs LightGBM comparison ──────────────────────────────
import time

X, y = make_classification(n_samples=10000, n_features=30, n_informative=15,
                           n_redundant=5, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

results = {}

try:
    import xgboost as xgb
    start = time.time()
    xgb_model = xgb.XGBClassifier(n_estimators=200, max_depth=6, learning_ra
                                  use_label_encoder=False, eval_metric='lo
    xgb_model.fit(X_train, y_train)
    results['XGBoost'] = {
        'accuracy': accuracy_score(y_test, xgb_model.predict(X_test)),
        'auc': roc_auc_score(y_test, xgb_model.predict_proba(X_test)[:, 1]),
        'time': time.time() - start,
    }
    print(f"XGBoost: acc={results['XGBoost']['accuracy']:.4f}, AUC={results[
except ImportError:
    print("XGBoost not installed: pip install xgboost")

try:
    import lightgbm as lgb
    start = time.time()
    lgb_model = lgb.LGBMClassifier(n_estimators=200, max_depth=6, learning_r
                                   random_state=42, verbose=-1)
    lgb_model.fit(X_train, y_train)
    results['LightGBM'] = {
        'accuracy': accuracy_score(y_test, lgb_model.predict(X_test)),
        'auc': roc_auc_score(y_test, lgb_model.predict_proba(X_test)[:, 1]),
        'time': time.time() - start,
    }
    print(f"LightGBM: acc={results['LightGBM']['accuracy']:.4f}, AUC={result
except ImportError:
    print("LightGBM not installed: pip install lightgbm")

# Compare with sklearn gradient boosting
start = time.time()
```

```python
gb_model = GradientBoostingClassifier(n_estimators=200, max_depth=6, learnir
gb_model.fit(X_train, y_train)
results['sklearn GB'] = {
    'accuracy': accuracy_score(y_test, gb_model.predict(X_test)),
    'auc': roc_auc_score(y_test, gb_model.predict_proba(X_test)[:, 1]),
    'time': time.time() - start,
}
print(f"sklearn GB: acc={results['sklearn GB']['accuracy']:.4f}, AUC={result
```

3. Handling Imbalanced Data

When 95% of samples are class 0, a model predicting "always 0" gets 95% accuracy but is useless.

Technique	Approach	When to Use
Class weights	Penalize mistakes on minority class more	Always try first
Threshold tuning	Move decision boundary from 0.5	When you need precision/recall tradeoff
SMOTE	Generate synthetic minority samples	When you have very few minority samples
Downsample majority	Remove majority samples	When you have tons of data

In [ ]:

```python
# —— Imbalanced data: techniques comparison ————————————————————

# Create imbalanced dataset (95:5 ratio)
X_imb, y_imb = make_classification(n_samples=5000, n_features=20, n_informat
                                   weights=[0.95], random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X_imb, y_imb, test_size=0.2, rando

print(f"Class distribution: {np.bincount(y_tr)}")
print(f"Minority class: {np.bincount(y_tr)[1] / len(y_tr) * 100:.1f}%\n")

# Method 1: Default (no handling)
clf_default = LogisticRegression(max_iter=1000)
clf_default.fit(X_tr, y_tr)

# Method 2: Class weights
clf_weighted = LogisticRegression(max_iter=1000, class_weight='balanced')
clf_weighted.fit(X_tr, y_tr)

# Method 3: Threshold tuning
y_proba = clf_default.predict_proba(X_te)[:, 1]
thresholds = np.arange(0.1, 0.9, 0.05)
f1_scores = [f1_score(y_te, (y_proba >= t).astype(int)) for t in thresholds]
best_threshold = thresholds[np.argmax(f1_scores)]

print(f"{'Method':<25} {'Accuracy':>10} {'F1':>10} {'Recall':>10}")
print("-" * 58)
```

```
for name, pred in [
    ('Default (threshold=0.5)', clf_default.predict(X_te)),
    ('Class weights=balanced', clf_weighted.predict(X_te)),
    (f'Best threshold={best_threshold:.2f}', (y_proba >= best_threshold).ast
]:
    print(f"{name:<25} {accuracy_score(y_te, pred):>10.4f} {f1_score(y_te, p
          f"{(pred[y_te==1].sum()/y_te.sum()):>10.4f}")

# Plot threshold tuning
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(thresholds, f1_scores, 'bo-')
ax.axvline(x=best_threshold, color='red', linestyle='--', label=f'Best thres
ax.set_xlabel('Classification Threshold')
ax.set_ylabel('F1 Score')
ax.set_title('Threshold Tuning: Finding the Optimal Decision Boundary')
ax.legend()
plt.tight_layout()
plt.show()
```

4. Hyperparameter Optimization with Optuna

Optuna uses **Bayesian optimization** (TPE sampler) instead of grid/random search:

- Learns which regions of hyperparameter space are promising
- 10-100x more efficient than grid search
- Built-in pruning: stops bad trials early

In [ ]:

```
# ── Optuna hyperparameter optimization ──────────────────────────

try:
    import optuna
    optuna.logging.set_verbosity(optuna.logging.WARNING)

    def objective(trial):
        params = {
            'n_estimators': trial.suggest_int('n_estimators', 50, 300),
            'max_depth': trial.suggest_int('max_depth', 3, 12),
            'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3,
            'min_samples_split': trial.suggest_int('min_samples_split', 2, 2
            'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10)
        }
        model = GradientBoostingClassifier(**params, random_state=42)
        scores = cross_val_score(model, X_train, y_train, cv=3, scoring='roc
        return scores.mean()

    study = optuna.create_study(direction='maximize')
    study.optimize(objective, n_trials=30, show_progress_bar=False)

    print(f"Best AUC: {study.best_value:.4f}")
    print(f"Best params: {study.best_params}")

    # Train with best params
```

```
    best_model = GradientBoostingClassifier(**study.best_params, random_stat
    best_model.fit(X_train, y_train)
    print(f"Test AUC: {roc_auc_score(y_test, best_model.predict_proba(X_test

except ImportError:
    print("Optuna not installed: pip install optuna")
    print("Using RandomizedSearchCV as fallback...")
    from sklearn.model_selection import RandomizedSearchCV
    from scipy.stats import randint, uniform

    param_dist = {
        'n_estimators': randint(50, 300),
        'max_depth': randint(3, 12),
        'learning_rate': uniform(0.01, 0.29),
    }
    search = RandomizedSearchCV(GradientBoostingClassifier(random_state=42),
                                param_dist, n_iter=20, cv=3, scoring='roc_au
    search.fit(X_train, y_train)
    print(f"Best AUC: {search.best_score_:.4f}")
    print(f"Best params: {search.best_params_}")
```

5. Model Interpretability with SHAP

SHAP (SHapley Additive exPlanations) answers "why did the model make this prediction?"

- Based on game theory: Shapley values = fair contribution of each feature
- Model-agnostic: works with any model
- Global AND local: understand the model overall and individual predictions

In [ ]:
```
# ── SHAP model interpretability ──────────────────────────────────

try:
    import shap

    # Train a model on California housing
    housing = fetch_california_housing(as_frame=True)
    X_h, y_h = housing.data, housing.target
    X_h_train, X_h_test = X_h[:1000], X_h[1000:1200]
    y_h_train, y_h_test = y_h[:1000], y_h[1000:1200]

    model_h = GradientBoostingClassifier if False else None
    from sklearn.ensemble import GradientBoostingRegressor
    model_h = GradientBoostingRegressor(n_estimators=100, max_depth=5, rando
    model_h.fit(X_h_train, y_h_train)

    # SHAP values
    explainer = shap.TreeExplainer(model_h)
    shap_values = explainer.shap_values(X_h_test)

    # Summary plot (feature importance)
    fig, ax = plt.subplots(figsize=(10, 6))
    shap.summary_plot(shap_values, X_h_test, show=False)
```

```
    plt.title('SHAP Feature Importance: California Housing')
    plt.tight_layout()
    plt.show()

except ImportError:
    print("SHAP not installed: pip install shap")
    print("\nAlternative: sklearn feature importance")

    housing = fetch_california_housing(as_frame=True)
    X_h, y_h = housing.data[:1000], housing.target[:1000]
    from sklearn.ensemble import GradientBoostingRegressor
    model_h = GradientBoostingRegressor(n_estimators=100, random_state=42)
    model_h.fit(X_h, y_h)

    feat_imp = pd.Series(model_h.feature_importances_, index=housing.feature
    feat_imp.sort_values().plot(kind='barh', figsize=(10, 5))
    plt.title('Feature Importance (built-in, less interpretable than SHAP)')
    plt.tight_layout()
    plt.show()
```

6. Production Pipelines with sklearn

Never do this in production:

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)  # Fit on train
X_test = scaler.transform(X_test)        # Transform test
separately
# What if someone forgets .transform() vs .fit_transform()?
```

Always use pipelines: transforms + model as a single, serializable unit.

In [ ]:

```
# ── Production sklearn pipeline ──────────────────────────────────
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

# Simulate a real dataset with mixed types
np.random.seed(42)
n = 1000
data = pd.DataFrame({
    'age': np.random.normal(40, 15, n),
    'income': np.random.lognormal(10, 1, n),
    'credit_score': np.random.normal(700, 50, n),
    'employment': np.random.choice(['full-time', 'part-time', 'self-employed
    'education': np.random.choice(['high-school', 'bachelors', 'masters', 'p
})
data.loc[np.random.choice(n, 50), 'income'] = np.nan  # Add missing values
target = ((data['income'].fillna(data['income'].median()) > 30000) &
          (data['credit_score'] > 680)).astype(int)

# Define column types
numeric_features = ['age', 'income', 'credit_score']
```

```
categorical_features = ['employment', 'education']

# Build pipeline
numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])

categorical_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='constant', fill_value='unknown')),
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features),
    ])

# Full pipeline: preprocessing + model
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', GradientBoostingClassifier(n_estimators=100, random_state
])

# Train and evaluate
X_tr, X_te, y_tr, y_te = train_test_split(data, target, test_size=0.2, rando
pipeline.fit(X_tr, y_tr)

print(f"Pipeline test accuracy: {pipeline.score(X_te, y_te):.4f}")
print(f"Pipeline steps: {[step[0] for step in pipeline.steps]}")
print(f"\nThis entire pipeline can be saved with joblib.dump(pipeline, 'mode
print("And loaded in production: pipeline.predict(new_data) — handles ALL pr
```

Key Takeaways

Concept	One-Line Summary
Learning curves	Train-val gap = variance; both low = bias
XGBoost/LightGBM	LightGBM is faster, XGBoost is more mature; both dominate tabular data
Imbalanced data	Class weights first, then threshold tuning
Optuna	Bayesian optimization > grid search; 10x fewer trials needed
SHAP	The gold standard for model interpretability
sklearn Pipelines	Pack preprocessing + model into one serializable object

What to study next:

- **Notebook 13**: CNNs and training techniques

- **Notebook 14**: GenAI and embeddings